Manuel d'utilisation — OpenHands

> Orchestrateur d'agents IA pour développement web métier complexe
> Instance locale : **http://localhost:3001**

Sommaire

1. Vue d'ensemble de votre setup
2. Démarrage et arrêt
3. Interface — les zones clés
4. Choisir le bon modèle selon la tâche
5. Stratégies de prompt pour le dev web métier
6. Workflow projet complexe
7. Donner accès à votre codebase
8. MCP — connecter vos outils métier
9. Cas d'usage concrets
10. Limites et pièges à éviter
11. Commandes de maintenance

1. Vue d'ensemble de votre setup

Composant	Valeur
URL	http://localhost:3001
Agent par défaut	`CodeActAgent`
LLM principal	DeepSeek v3 (code, raisonnement)
LLM chat/condenser	Claude Sonnet 4.5
LLM multimodal	Gemini Flash 1.5
LLM local	Gemma 4 via Ollama (localhost:11434)
Provider API	OpenRouter
Workspace monté	`./workspace`
Config	`config.toml`

Architecture OpenHands :

```
Navigateur (3001)
    └── OpenHands App (FastAPI/Uvicorn)
            ├── CodeActAgent  ← exécute du code dans un sandbox Docker
            ├── LLM Router    ← DeepSeek v3 par défaut via OpenRouter
            └── Sandbox       ← conteneur isolé par session
```

L'agent **voit et exécute** : terminal bash, éditeur de fichiers, navigateur headless, Jupyter.
Il **ne voit pas** votre machine hôte sauf ce que vous montez dans `workspace/`.

---

2. Démarrage et arrêt

```bash
# Démarrer
cd ~/Depot/www/OpenHandHarnai
docker compose up -d

# Vérifier que c'est bien lancé
docker compose ps

# Voir les logs en temps réel
docker compose logs -f

# Arrêter proprement
docker compose down

# Redémarrer après modification de config.toml
docker compose down && docker compose up -d
```

> **Après un reboot machine :** OpenHands ne redémarre pas automatiquement.
> Ajoutez `restart: unless-stopped` dans `docker-compose.yml` si vous voulez un démarrage auto.

---

3. Interface — les zones clés

```
┌──────────────────────────────────────────────────┐
│  [Nouvelle conversation]    [Historique sessions]  │
├──────────────┬───────────────────────────────────┤
│              │                                   │
│  Explorateur │   Zone de conversation            │
│  de fichiers │                                   │
│  (workspace) │   > Votre prompt                  │
│              │   < Réponse agent + actions        │
│              │                                   │
├──────────────┴───────────────────────────────────┤
│  [Terminal intégré]  [Éditeur]  [Navigateur]      │
└──────────────────────────────────────────────────┘
```

Onglets du bas :

- **Terminal** — vous pouvez voir et intercepter les commandes que l'agent exécute

- **Éditeur** — vue live des fichiers modifiés
- **Navigateur** — l'agent peut naviguer sur des URL, tester des interfaces

Paramètres de session (icône ⚙️) :

- Changer le modèle LLM pour cette session
- Activer le mode confirmation (l'agent demande avant d'exécuter)
- Définir le répertoire de travail

---

4. Choisir le bon modèle selon la tâche

Changez le modèle dans les paramètres ⚙️ de chaque session.

Tâche	Modèle recommandé	Pourquoi
Écrire du code métier complexe	**DeepSeek v3** (défaut)	Meilleur rapport qualité/coût pour le code
Concevoir une architecture	**Claude Sonnet**	Raisonnement long, mémoire de contexte 200k
Analyser une maquette/screenshot	**Gemini Flash 1.5**	Multimodal natif
Tests unitaires en masse	**DeepSeek v3**	Rapide et précis
Tâche sensible / code privé	**Gemma 4 local**	Aucune donnée sortante
Refactoring large base de code	**Claude Sonnet**	Contexte long, cohérence
Debug rapide / question ponctuelle	**Gemini Flash 1.5**	Très rapide, bon marché

> **Astuce :** Pour les projets où la confidentialité est critique, basculez sur `gemma_local` — tout tourne sur votre machine, zéro donnée envoyée.

---

5. Stratégies de prompt pour le dev web métier

5.1 Prompt de contexte projet (à mettre en début de session)

Collez ce bloc en début de chaque nouvelle session importante :

```
Contexte projet :
- Stack : [ex: Django + PostgreSQL + React / Vue / HTMX]
- Domaine métier : [ex: gestion CMDB, facturation, logistique]
- Conventions : [ex: snake_case Python, PEP8, tests pytest, migrations Alembic]
- Répertoire de travail : /workspace/[nom-projet]
- Ne jamais modifier les fichiers de config production directement.

Objectif de cette session : [description précise]
```

5.2 Prompt de tâche efficace

Mauvais prompt :

```
Fais-moi une API pour les utilisateurs.
```

Bon prompt :

```
Dans /workspace/monprojet/api/users/, crée un endpoint REST POST /users/
- Valide email unique + mot de passe >= 12 chars
- Hash bcrypt, stocke en base (modèle User existant dans models.py)
- Retourne JWT + refresh token
- Écris les tests dans tests/test_users.py
- Respecte le style des endpoints existants dans api/auth/
```

5.3 Phrases de contrôle utiles

```
"Montre-moi le plan avant d'écrire du code."
"Ne modifie que ce fichier, rien d'autre."
"Explique chaque décision technique avant d'implémenter."
"Arrête-toi après l'étape 2 et attends ma validation."
"Regarde d'abord le code existant dans [fichier] avant de proposer quoi que ce soit."
```

5.4 Mode itératif (recommandé pour le métier complexe)

```
Étape 1 : Analyse le schéma de base de données dans /workspace/db/schema.sql
          et liste les entités principales. Ne code rien encore.

[validation]

Étape 2 : Propose l'architecture des services pour [fonctionnalité].
          Une classe par fichier, détaille les interfaces.

[validation]

Étape 3 : Implémente uniquement le service [X], avec ses tests.
```

---

6. Workflow projet complexe

6.1 Démarrage d'un nouveau chantier

```
1. Monter le projet dans workspace/          (voir section 7)
2. Ouvrir une session OpenHands
3. Prompt de contexte projet                 (voir 5.1)
4. Demander une lecture + synthèse du code   ("Lis le projet et résume l'architecture")
5. Valider la compréhension de l'agent
6. Procéder tâche par tâche
```

6.2 Revue de code par l'agent

```
Lis tous les fichiers dans /workspace/monprojet/api/
Identifie :
1. Les failles de sécurité éventuelles
2. Les violations des conventions DRY/SOLID
3. Les requêtes N+1 potentielles
4. Les endpoints sans validation d'entrée
Produis un rapport structuré, sans modifier de fichier.
```

6.3 Migration / refactoring

```
Je veux migrer [module X] de [ancienne techno] vers [nouvelle techno].
Étapes attendues :
1. Inventaire des dépendances actuelles
2. Plan de migration avec risques identifiés
3. Implémentation fichier par fichier avec tests
4. Script de validation post-migration
Commence par l'étape 1 seulement.
```

6.4 Debug d'un bug métier

```
Bug : [description du comportement observé]
Reproductible : [étapes pour reproduire]
Logs : [coller les logs ou indiquer le chemin]
Fichiers suspects : [liste si connue]

Commence par lire les logs et identifier la cause racine.
Ne propose pas de fix avant d'avoir confirmé la cause.
```

---

7. Donner accès à votre codebase

Option A — Lien symbolique dans workspace (recommandé)

```
# Depuis votre machine hôte
ln -s ~/Depot/www/MonProjet ~/Depot/www/OpenHandHarnai/workspace/MonProjet
```

L'agent y accède via `/workspace/MonProjet`.

Option B — Volume supplémentaire dans docker-compose.yml

```
volumes:
  - /var/run/docker.sock:/var/run/docker.sock
  - ~/.openhands:/.openhands
  - ${WORKSPACE_BASE:-$PWD/workspace}:/opt/workspace_base
  - /home/django/Depot/www:/workspace/projets   # ← ajout
```

```
docker compose down && docker compose up -d
```

L'agent accède alors à tous vos projets via `/workspace/projets/`.

Option C — Copie ponctuelle

```
cp -r ~/Depot/www/MonProjet ~/Depot/www/OpenHandHarnai/workspace/
```

> **Important :** Les modifications faites par l'agent dans `/workspace/` sont **persistantes** sur votre disque. Faites un `git commit` avant chaque session importante pour pouvoir revenir en arrière.

---

8. MCP — connecter vos outils métier

Le protocole MCP permet à l'agent d'interagir avec vos outils externes (base de données, API, fichiers système).

Exemple : accès filesystem étendu

Dans `config.toml`, section `[mcp]` :

```
[mcp]
stdio_servers = [
    {name = "filesystem", command = "npx", args = ["@modelcontextprotocol/server-filesystem", "/home/django/Depot"]}
]
```

Exemple : base de données PostgreSQL

```
[mcp]
stdio_servers = [
    {name = "postgres", command = "uvx", args = ["mcp-server-postgres",
"postgresql://user:pass@localhost/mabase"]}
]
```

L'agent peut alors faire des requêtes SQL directement pendant vos sessions.

> Après modification de `config.toml` : `docker compose down && docker compose up -d`

---

9. Cas d'usage concrets

Générer un module CRUD complet

```
Dans /workspace/monprojet/, génère un module CRUD complet pour l'entité "Contrat" :
- Modèle SQLAlchemy avec champs : id, référence (unique), client_id (FK), montant, statut,
date_début, date_fin
- Migration Alembic
- Service avec méthodes : create, get_by_id, list (paginé, filtrable par statut), update,
soft_delete
- Router FastAPI avec validation Pydantic
- Tests pytest couvrant les cas nominaux et les erreurs métier
Respecte la structure du module "Facture" déjà présent dans
/workspace/monprojet/modules/facture/
```

Analyser une maquette UI (Gemini Flash)

```
[joindre screenshot de la maquette]
Implémente ce formulaire en [React/Vue/Django template].
Valide les champs côté client ET côté serveur.
Les champs obligatoires sont marqués *.
```

Écrire des tests sur du code existant

```
Lis /workspace/monprojet/services/facturation.py
Écris des tests pytest exhaustifs dans tests/test_facturation.py :
- Couvre tous les chemins de code (branches if/else)
- Teste les cas limites métier (montant nul, date passée, client inactif)
- Mock les appels externes (Stripe, email)
- Vise 90%+ de couverture sur ce fichier
```

Documenter une API existante

```
Lis tous les routers dans /workspace/monprojet/api/
Génère la documentation OpenAPI complète en YAML :
- Descriptions des endpoints
- Schémas de requête et réponse
- Codes d'erreur avec exemples
- Exemples de payload réels tirés des tests existants
```

---

10. Limites et pièges à éviter

Ce que l'agent fait mal

Situation	Risque	Mitigation
Sessions très longues (>50 échanges)	Perd le contexte métier	Ouvrir une nouvelle session, redonner le contexte
Fichiers >500 lignes	Lecture partielle	Découper, indiquer les lignes précises
Modifications sur plusieurs fichiers à la fois	Erreurs de cohérence	Valider fichier par fichier
Logique métier implicite	Invente des règles	Toujours expliciter les règles métier
Base de données de production	Risque de données	Ne jamais pointer sur la prod

Règles d'hygiène

```
  Toujours faire un git commit avant une session de modification
  Activer le mode confirmation pour les sessions à risque (⚙️ > Confirmation mode)
  Relire chaque diff avant de valider
  Tester dans le sandbox avant de copier en prod
  Ne pas donner de credentials de production dans les prompts
  Ne pas laisser l'agent modifier les fichiers de migration directement en prod
  Ne pas utiliser DeepSeek/Claude pour du code contenant des données personnelles sensibles
→ utiliser Gemma local
```

Récupérer après une session qui a mal tourné

```bash
# Voir ce qui a changé dans workspace
cd ~/Depot/www/OpenHandHarnai/workspace/MonProjet
git diff

# Annuler les modifications non voulues
git checkout -- .

# Ou revenir au dernier commit propre
git reset --hard HEAD
```

11. Commandes de maintenance

```
# Statut des conteneurs
docker compose ps

# Logs en temps réel
docker compose logs -f openhands

# Redémarrer après modif config
docker compose down && docker compose up -d

# Mettre à jour OpenHands (nouvelle version)
git pull
docker compose down
docker compose build --no-cache
docker compose up -d

# Voir l'espace disque utilisé par Docker
docker system df

# Nettoyer les images/conteneurs inutiles
docker system prune -f

# Accéder au shell du conteneur OpenHands
docker exec -it openhands-app- bash

# Vérifier que l'API OpenRouter fonctionne
curl https://openrouter.ai/api/v1/models \
  -H "Authorization: Bearer $(grep api_key ~/Depot/www/OpenHandHarnai/config.toml | head -1
| cut -d'"' -f2)" \
  | python3 -m json.tool | grep '"id"' | head -10
```

Référence rapide — Modèles OpenRouter disponibles

ID dans config.toml	Modèle réel	Forces
`openrouter/deepseek/deepseek-chat-v3-0324`	DeepSeek v3	Code, raisonnement, très bon marché
`openrouter/anthropic/claude-sonnet-4-5`	Claude Sonnet 4.5	Contexte 200k, cohérence longue session
`openrouter/google/gemini-flash-1.5`	Gemini Flash 1.5	Multimodal, très rapide
`ollama/gemma3:27b`	Gemma 4 local	Privé, hors ligne

> Pour voir tous les modèles disponibles sur OpenRouter : https://openrouter.ai/models

Manuel généré pour votre instance OpenHands v1.29.0 — mis à jour le 29/06/2026